Chapter 18: Trees

EECS 281 notes, condensed. Covers 18.1 to 18.9.

Legend

- **Red box** = section marked ✻ in the textbook. Treated here as optional / lower priority (check your syllabus). These are:
  - 18.8.3 `std::multimap` / `std::multiset` (incl. the multiset streaming median)
  - 18.9 Tries, including 18.9.1–18.9.5

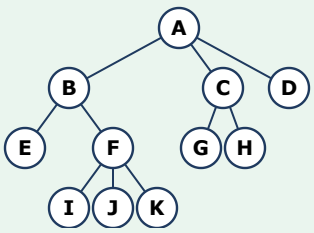
  Everything else (18.1–18.8.2) is unmarked core material.
- ★ = core, high-yield material worth memorizing.
- Boxes: **blue** = definition, **green** = worked example, **yellow** = remark / watch out. In diagrams: green = new node, red = imbalanced / target node, **±2** = balance factor.

18.1 Introduction to Trees

Graph: nodes connected by edges. **Tree:** a restricted graph (every tree is a graph, not every graph is a tree).
Simple tree: an **acyclic** (no two nodes joined by more than one route) and **connected** (a path between any two nodes) graph.
Rooted tree: a simple tree with a special **root** node; all edges branch away from it (any node can be picked as root).
Ordered tree: children of each node have a fixed linear order (e.g. left, right). Swapping siblings gives a *different* ordered tree but the *same* unordered tree.

Term	Meaning
Root	Top-most node; everything is directed away from it.
Parent / child	For each edge, the higher node is the parent, the lower is the child.
Sibling	Nodes that share a parent.
Descendant	Any node on a path from the node down to a leaf (including the leaf).
Ancestor	Any node on the path from the node up to the root (including the root).
External (leaf) node	No children.
Internal node	Has at least one child.
★ Depth	Distance from the top. **Root has depth 1.** depth(empty) = 0, depth(node) = depth(parent) + 1.
★ Height	Distance from the bottom. **Leaf has height 1.** height(empty) = 0, height(node) = max(height(children)) + 1.

The largest height in a tree equals the largest depth (= number of levels).



Example 18.1 Root = A. Internal = A, B, C, F. Leaves = D, E, G, H, I, J, K. Max depth = 4 (number of levels). height(B) = max(height(E), height(F)) + 1 = max(1, 2) + 1 = **3**.

18.2 Binary Trees

Binary tree: an ordered tree where each node has **at most two children** (left and right). Two ways to store one: array-based or pointer-based.

18.2.1 Array-based implementation

Flatten the tree level by level, same as a binary heap (1-indexed):

- Root at index **1**. Left child of i at $\mathbf{2i}$, right child at $\mathbf{2i + 1}$, parent at $\mathbf{i/2}$ (all $\Theta(1)$ arithmetic). Missing child → that index is left empty.
- Root at index 0 also works if you adjust the formulas (e.g. children $2i+1$, $2i+2$).
- Great for **complete** trees (all levels full except the bottom, which fills left to right): no gaps. That is why heaps use arrays.
- Bad for **sparse** trees: every missing node still costs a slot. A **stick** (one node per level) of n nodes needs depth n, i.e. all slots up to level n.

★ Max nodes at depth $d = 2^{d-1}$, so a stick needs $2^0 + 2^1 + \ldots + 2^{n-1} = 2^n - 1 = \mathbf{\Theta(2^n)}$ space. In general, with up to k children per node (geometric series $a(1 - r^n)/(1 - r)$): $(k^n - 1)/(k - 1) = \mathbf{\Theta(k^n)}$. E.g. $k = 3 \rightarrow \frac{1}{2}(3^n - 1) = \Theta(3^n)$.

> **Example 18.2** Array-based tree, up to 10 children per node, n nodes: best case (no gaps) $\Theta(n)$ space; worst case (stick) $\Theta(10^n)$.

18.2.2 Pointer-based implementation

```
template <typename T>
struct Node {
  T val;
  Node* left;    // nullptr if no left child
  Node* right;   // nullptr if no right child
};
```

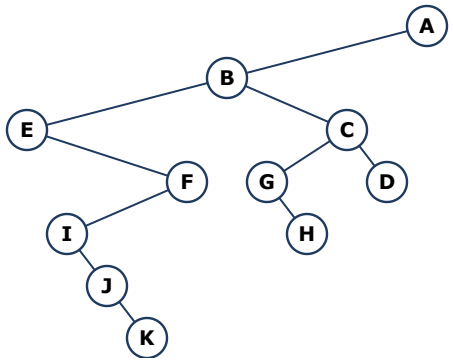
Memory is allocated only for nodes that exist → always $\Theta(n)$ space.

Optional `Node* parent` (nullptr for root) makes find-parent $\Theta(1)$, but it is rarely used: costs memory, and **recursion emulates it** (unrolling a call moves you back to the parent).

Operation (n nodes)	Array-based	Pointer-based	Why
Insert key, best case	$\Theta(1)$	$\Theta(1)$	Root has an open child slot.
Insert key, worst case	$\Theta(n)$	$\Theta(n)$	No ordering rule: may scan everything for an open spot (array may also need to resize).
Remove key, worst case	$\Theta(n)$	$\Theta(n)$	Key could be anywhere, or absent (must check every node).
Find parent	$\Theta(1)$	★ $\Theta(n)$	Pointer: must search for the node whose child it is ($\Theta(1)$ with a parent pointer).
Find child	$\Theta(1)$	$\Theta(1)$	Index math / stored pointers.
Space, best case	$\Theta(n)$	$\Theta(n)$	Complete tree.
Space, worst case	★ $\Theta(2^n)$	$\Theta(n)$	Stick.

18.2.3 Converting a generic tree into a binary tree

For a node v with children $v_1, \ldots, v_k$: (1) make v_1 the **left** child of v; (2) chain $v_2, \ldots, v_k$ as successive **right** children of v_1; (3) recurse. **Left = first child, right = next sibling**, exactly like a pairing heap.



Example 18.1's tree converted (left = first child, right = next sibling)

> **Example 18.3** Build the binary version of a `GenericNode{T val; vector<GenericNode*> children;}` tree.

```
template <typename T>
BinaryNode<T>* convert_generic_to_binary(GenericNode<T>* g) {
  return helper(g, new BinaryNode<T>{ g->val });
}
template <typename T>
BinaryNode<T>* helper(GenericNode<T>* g, BinaryNode<T>* b) {
  if (g->children.empty()) return b;                        // base case: no children
  GenericNode<T>* first = g->children[0];
  BinaryNode<T>* next_b = new BinaryNode<T>{ first->val };
  b->left = helper(first, next_b);                          // first child → left
  BinaryNode<T>* curr = next_b;
  for (size_t i = 1; i < g->children.size(); ++i) {       // other children → right chain
    curr->right = helper(g->children[i], new BinaryNode<T>{ g->children[i]->val });
    curr = curr->right;
  }
  return b;
}
```

18.3 Tree Traversals

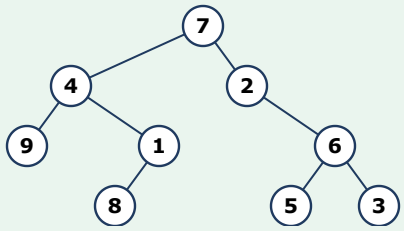
Tree traversal: a systematic way to process every node. The prefix says **when the parent (current node) is processed** relative to its children: **pre** = before, **in** = between, **post** = after. Pre/in/post are recursive; level-order is iterative (queue).

```
void preorder(Node* p) {
  if (!p) return;
  process(p->val);    // P
  preorder(p->left);  // L
  preorder(p->right); // R
}
```

```
void inorder(Node* p) {
  if (!p) return;
  inorder(p->left);   // L
  process(p->val);    // P
  inorder(p->right);  // R
}
```

```
void postorder(Node* p) {
  if (!p) return;
  postorder(p->left);  // L
  postorder(p->right); // R
  process(p->val);     // P
}
```

`process()` is a placeholder for the per-node work, e.g. `std::cout << p->val`.



Examples 18.4–18.7 (same tree)

Traversal	Order
Preorder (P L R)	7, 4, 9, 1, 8, 2, 6, 5, 3
Inorder (L P R)	9, 4, 8, 1, 7, 2, 5, 6, 3
Postorder (L R P)	9, 8, 1, 4, 5, 3, 6, 2, 7
Level-order	7, 4, 2, 9, 1, 6, 8, 5, 3

Hand-trace trick: label each node with its three steps (e.g. P L R) and tick each off as the recursion completes it; a step can only start once the one before it is done. ★ Preorder always starts with the root; postorder always ends with it.

18.3.4 Level-order traversal

Process nodes by increasing depth, left to right within a level. Uses a **queue** (FIFO) instead of recursion: this is breadth-first search (next chapter).

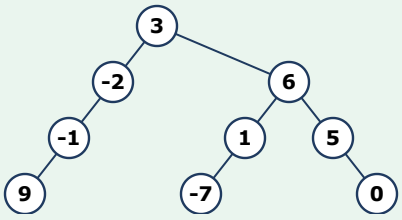
```
void level_order(Node* root) {
  if (!root) return;
  std::queue<Node*> bfs;
  bfs.push(root);
  while (!bfs.empty()) {
    Node* curr = bfs.front();
    bfs.pop();
    process(curr);
    if (curr->left)  bfs.push(curr->left);
    if (curr->right) bfs.push(curr->right);
  }
}
```

Each iteration: pop front, process it, push its left then right child (if they exist).

★ **Level-by-level trick:** right after a whole level has been processed, `bfs.size()` = number of nodes in the *next* level. Read the size at the top of the loop and process exactly that many nodes in an inner loop.

Trace on the tree above: queue [7] → [4, 2] → [2, 9, 1] → [9, 1, 6] → … → [] (9 iterations).

Example 18.8: maximum level sum. Return the largest sum of any single level.



sums 3, 4, **5**, 2 → return 5

```
int32_t max_level_sum(Node* root) {
  int32_t curr_max = std::numeric_limits<int32_t>::min();
  std::queue<Node*> bfs;
  bfs.push(root);
  while (!bfs.empty()) {
    size_t level_size = bfs.size();     // nodes in this level
    int32_t level_sum = 0;
    for (size_t i = 0; i < level_size; ++i) {
      Node* curr = bfs.front(); bfs.pop();
      level_sum += curr->val;
      if (curr->left)  bfs.push(curr->left);
      if (curr->right) bfs.push(curr->right);
    }
    curr_max = std::max(curr_max, level_sum);
  }
  return curr_max;
}
```

18.4 Solving Problems Using Trees

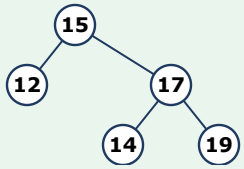
Trees are recursive, so most tree problems follow a ★ **four-step recipe**:

1. **Base case:** input you can answer trivially (usually `root == nullptr`).
2. Recursively solve the **left** subtree.
3. Recursively solve the **right** subtree.
4. **Combine**: use the left/right results to solve for the current node.

Step 4 **before** 2 and 3 = preorder style (need the node first). Step 4 **after** = postorder style (need the children's answers first, i.e. bottom-up).

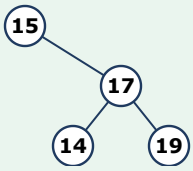
Example 18.9: max height (postorder). Empty tree = 0; height = 1 + max(L, R).

```
int32_t find_max_height(Node* root) {
  if (!root) return 0;                          // 1. base
  int32_t l = find_max_height(root->left);      // 2.
  int32_t r = find_max_height(root->right);     // 3.
  return 1 + std::max(l, r);                    // 4. combine
}
```



height 3 · min leaf depth 2 · left-leaf sum 12 + 14 = 26

Example 18.10: minimum leaf depth (root depth 1). Naive 1 + min(L, R) is **wrong**: a `nullptr` child returns 0, and nullptr is not a leaf.



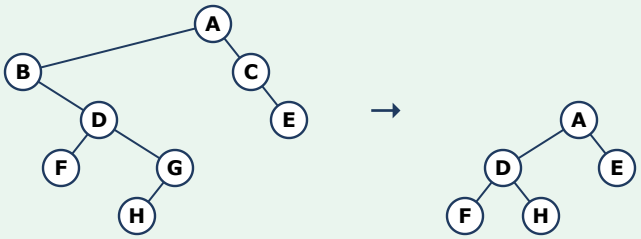
naive gives 1, correct is 3

```
int32_t find_min_depth(Node* root) {
  if (!root) return 0;
  int32_t l = find_min_depth(root->left);
  int32_t r = find_min_depth(root->right);
  if (l == 0 || r == 0)                 // one side missing:
    return 1 + std::max(l, r);          //   use the side that exists
  return 1 + std::min(l, r);
}
```

Example 18.11: sum of all left leaves (postorder). If the left child is a leaf, use its value plus the right subtree's sum; otherwise add both subtrees' sums.

```
int32_t left_leaf_sum(Node* root) {
  if (!root) return 0;
  int32_t sum_l = left_leaf_sum(root->left);
  int32_t sum_r = left_leaf_sum(root->right);
  if (root->left && !root->left->left && !root->left->right)   // left child is a leaf
    return root->left->val + sum_r;
  return sum_l + sum_r;
}   // optimization: only recurse left when the left child is not a leaf
```

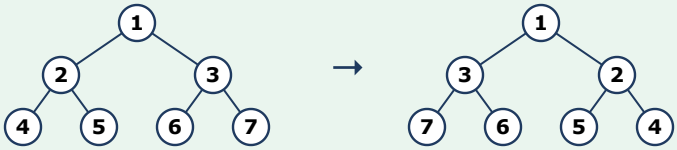
Example 18.12: make a tree full. A **full (proper)** binary tree: every node has 0 or 2 children. Remove every node with exactly one child. Must handle children before the node itself → **postorder**.



remove B, C, G

```
Node* remove_single_children(Node* root) {
  if (!root) return nullptr;
  root->left  = remove_single_children(root->left);
  root->right = remove_single_children(root->right);
  bool none = !root->left && !root->right;
  bool both = root->left && root->right;
  if (none || both) return root;   // keep 0/2-child node
  Node* child = root->left ? root->left : root->right;
  delete root;                     // (book omits this; leaks)
  return child;                    // splice child up to parent
}
```

Example 18.13: invert (mirror) a tree. Swap each node's children, then recurse (preorder; postorder works too).

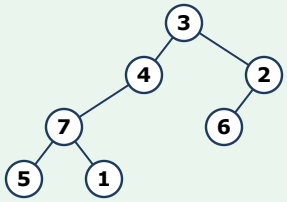



```
void invert_binary_tree(Node* root) {
  if (!root) return;
  std::swap(root->left, root->right);
  invert_binary_tree(root->left);
  invert_binary_tree(root->right);
}
```

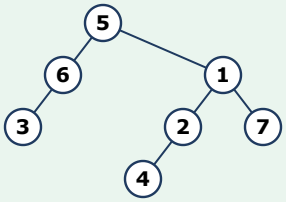
18.5 Binary Tree Traversal Reconstruction

★ **Inorder + (preorder or postorder)** → a **unique** binary tree. Pre/post tells you the **root** of each subtree (first in preorder, last in postorder); inorder **partitions** the rest: everything before the root in inorder is in the left subtree, everything after is in the right. Repeat on each side.
Preorder + postorder only → **not** unique: you can't tell whether a lone child is a left or right child.

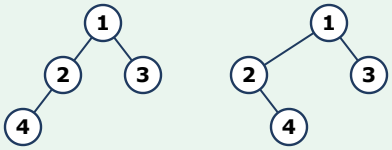
Ex 18.14 In: 5 7 1 4 **3** 6 2 · Pre: **3** 4 7 5 1 2 6
Root 3 → left {5,7,1,4}, right {6,2}. Left root = 4 (first in pre); 5,7,1 all left of 4. Then 7 with 5 left, 1 right. Right: 2 is root, 6 left of it.



Ex 18.15 In: 3 6 **5** 4 2 1 7 · Post: 3 6 4 2 7 1 **5**
Root 5 (last in post) → left {3,6}, right {4,2,1,7}. Left root 6 (last of those in post), 3 left. Right root 1; {4,2} left, 7 right; 2 is root, 4 left.



Ex 18.16 Two different trees, same pre (1 2 4 3) and post (4 2 3 1): 4 is the left vs right child of 2.



18.6 Binary Search Trees

18.6.1 Structure

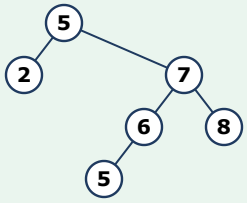
Binary search tree (BST): a binary tree ordered by a sorting invariant. For every node: (1) key **>** every key in its **left** subtree; (2) key **≤** every key in its **right** subtree; (3) both subtrees are BSTs too.

Check the **whole subtree**, not just the children: in `11(7(2, 13), 17)`, 13 is fine relative to 7 but is in 11's left subtree → not a BST. `11(7(2, 6), …)` fails because 6 < 7 is a right child.

Same keys can form **different** valid BSTs (shape depends on insertion order).

★ **Inorder traversal of a BST visits keys in sorted order.** So printing a BST in sorted order is $\Theta(n)$ (Example 18.18).

Example 18.17



inorder: 2, 5, 5, 6, 7, 8

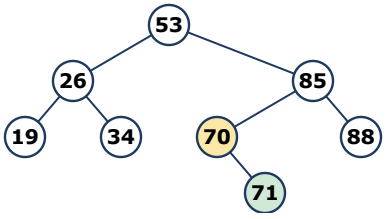
Duplicates: any consistent side works. Sending equal keys **right** means you only need `operator<` (if `k < node` go left, else go right), which is what STL containers use. Alternative: store a **count** per key (e.g. `5(2)`): height no longer grows with duplicates, and some balancing schemes get simpler. Not used here unless stated.

18.6.2 Searching

Compare the target *k* to the current node: smaller → go left, larger → go right, equal → found; hit `nullptr` → not present. Each step discards one subtree ("binary search").

```
template <typename T>   // iterative
Node<T>* tree_search(Node<T>* root, T k) {
  while (root && k != root->val)
    root = (k < root->val) ? root->left : root->right;
  return root;                    // nullptr if absent
}
```

```
template <typename T>   // recursive (tail)
Node<T>* tree_search(Node<T>* root, T k) {
  if (!root || root->val == k) return root;
  if (k < root->val) return tree_search(root->left, k);
  return tree_search(root->right, k);
}
```



Search 70: 70 > 53 → right; 70 < 85 → left; found.
Search 71: … 71 > 70 → right → nullptr → **not found**.
Insert 71 (18.6.3): same path, put the new node at that nullptr.

★ Cost = **height** of the tree. Best Θ(1) (root). Average Θ(log *n*) (roughly balanced, half discarded per level; average height is Θ(log *n*)). Worst Θ(*n*) (a **stick**).

18.6.3 Inserting

```
template <typename T>
struct Node {
  T val; Node* left; Node* right;
  Node(T k) : val{k}, left{nullptr}, right{nullptr} {}
};
template <typename T>
void tree_insert(Node<T>*& root, T k) {   // pointer BY REFERENCE
  if (root == nullptr) root = new Node<T>{k};
  else if (k < root->val) tree_insert(root->left, k);
  else tree_insert(root->right, k);    // equal keys go right
}
```

Walk down like search until you reach a `nullptr`, and attach there. ★ `Node*& root`: passing the pointer **by reference** lets `root = new Node` update the parent's `left`/`right` pointer directly. Best Θ(1), average Θ(log *n*), worst Θ(*n*) (stick).

Example 18.19: smallest key. Keep going left until there is no left child. Average Θ(log *n*), worst Θ(*n*) (left-facing stick).

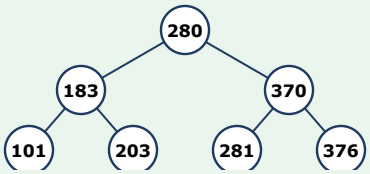
```
Node* bst_tree_min(Node* root) {
  if (!root) return nullptr;
  while (root->left) root = root->left;
  return root;
}
```

Example 18.20: how many insertion orders of *n* unique keys make a stick? (stick = no node has a sibling; worst case Θ(*n*) search/insert/remove). Not just sorted / reverse-sorted: 1, 4, 2, 3 is a zig-zag stick too. Rule: a stick forms iff **every insert is the smallest or largest remaining key**. 2 choices for each of the first $n-1$ inserts, 1 for the last → ★ **2^{n-1} stick orders** (8 for $n = 4$).

How many distinct BSTs from *n* unique keys? Not *n*! (that counts insertion orders, and e.g. 2,1,3 and 2,3,1 give the same tree). Pick key *i* (0-indexed) as root: *i* keys go left, $n - i - 1$ go right: $C_0 = C_1 = 1, \quad C_n = \sum_{i=0}^{n-1} C_i \cdot C_{n-i-1}$ closed form $C_n = (1/(n+1)) \cdot (2n \text{ choose } n) = (2n)! / ((n+1)!\, n!)$ (the **Catalan numbers**).

Example 18.21 7 keys: 101, 183, 203, 280, 281, 370, 376.

- Distinct BSTs: C_7 = 14! / (8! 7!) = **429**.
- Sticks: 2^6 = 64 → 64/429 = **14.92%**.
- Complete: only **one** shape is complete for any n → 1/429 = **0.23%**.



Example 18.22: sorted array → minimum-height BST in Θ(n). The middle element is the root; the middle of each half becomes each child.

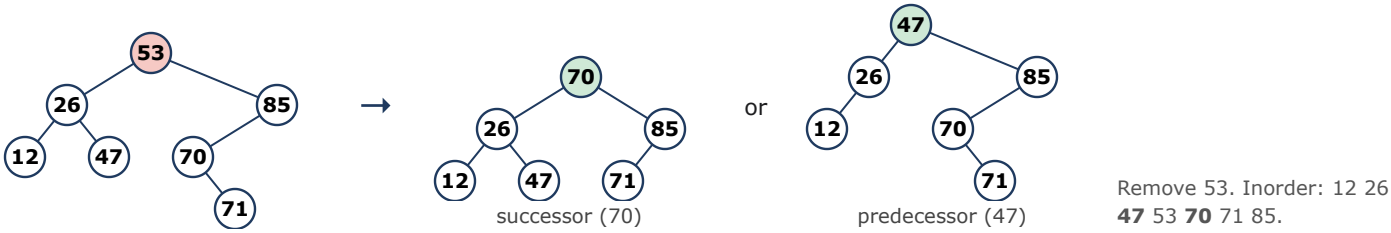
```
Node* array_to_bst(int32_t arr[], size_t left, size_t right) {   // range [left, right]
  if (left > right) return nullptr;
  size_t mid = left + (right - left) / 2;
  Node* root = new Node(arr[mid]);
  root->left  = array_to_bst(arr, left, mid - 1);
  root->right = array_to_bst(arr, mid + 1, right);
  return root;
}
```

Watch out: with `size_t`, `mid - 1` wraps around when `mid == 0`, so the base case never triggers. Use signed indices (or a half-open range) in real code.

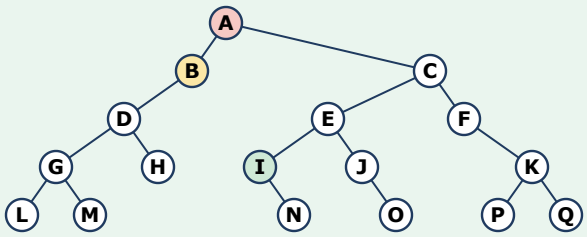
18.6.4 Removing

Node to remove has…	Do this
No children (leaf)	Just snip it off.
No left child	Replace it with its **right** child.
No right child	Replace it with its **left** child.
★ Two children	Copy in its **inorder successor** (smallest key in the right subtree) **or inorder predecessor** (largest in the left subtree), then delete that node from its subtree. These keys sit right next to the removed key in sorted order, so the BST property holds with no other changes.

★ **Inorder successor:** go **right once**, then **left as far as possible**. **Inorder predecessor:** go **left once**, then **right as far as possible**. They come directly after / before the node in an inorder traversal.



Remove 53. Inorder: 12 26 **47** 53 **70** 71 85.



Example 18.23 Successor of A: right to C, then left to E, then left to I; I has no left child → **I**.
Predecessor of A: left to B; B has no right child → **B**.

```
template <typename T>
void tree_remove(Node<T>*& root, const T& val) {
  if (root == nullptr) return;                                  // not found
  if (val < root->val)      tree_remove(root->left, val);
  else if (root->val < val) tree_remove(root->right, val);
  else {                                                        // found it
    Node<T>* node_to_delete = root;
    if (root->left == nullptr)       { root = root->right; delete node_to_delete; }
    else if (root->right == nullptr) { root = root->left;  delete node_to_delete; }
    else {                                                      // two children
```

```
      Node<T>* succ = root->right;
      while (succ->left) succ = succ->left;
      root->val = succ->val;                          // copy successor's value
      tree_remove(root->right, succ->val);            // delete the original successor
    }
  }
}
```

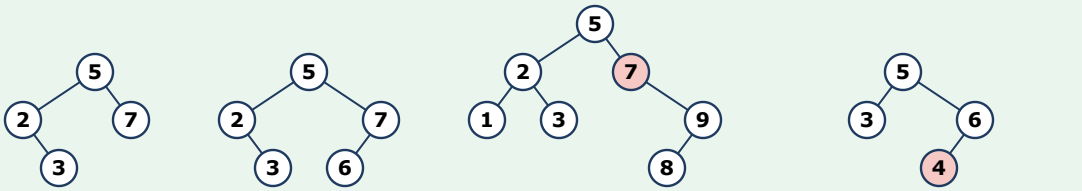
BST operation	Best	Average	Worst
Find / Insert / Delete a value	Θ(1)	Θ(log n)	★ Θ(n) (stick)

18.7 AVL Trees

18.7.1 Properties

AVL tree (Adelson-Velsky & Landis): a **self-balancing BST** enforcing the **height balance property**: at every node, the heights of the left and right subtrees differ by **at most 1**. If an operation breaks this, the tree fixes itself with **rotations** → ★ **worst-case Θ(log *n*)** search, insert and remove. (Height: leaf = 1, empty = 0.)

Example 18.24 Which are AVL trees?



Example 18.25: AVL height is O(log *n*). Let n_h = minimum nodes in an AVL tree of height h: $n_0 = 0$, $n_1 = 1$, and $n_h = 1 + n_{h-1} + n_{h-2}$ (root + the thinnest legal subtrees, heights $h-1$ and $h-2$). Since $n_{h-1} > n_{h-2}$: $n_h > 2n_{h-2} > 4n_{h-4} > \dots > 2^i n_{h-2i}$. Take $i = h/2 - 1$ (reach $n_1 = 1$): $n_h > 2^{h/2-1}$ → $\boldsymbol{h < 2\log(n_h) + 2 = O(\log n)}$.

Example 18.26: AVL tree of height 9.

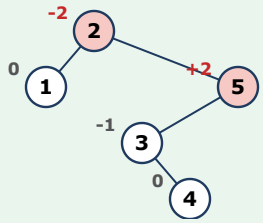
h	0	1	2	3	4	5	6	7	8	9
min nodes n_h	0	1	2	4	7	12	20	33	54	**88**

Min = **88**. Max = full tree = $2^h - 1 = 2^9 - 1 =$ **511**.

18.7.2 Balance factor

```
struct Node {
  int val;
  int height;          // stored in each node
  Node* left; Node* right;
  int left_height()  { return left  ? left->height  : 0; }
  int right_height() { return right ? right->height : 0; }
};
```

★ **balance factor** = `left_height() - right_height()`
Balanced iff every node's BF ∈ {−1, 0, +1}. +1 = left taller by 1; −1 = right taller by 1. |BF| > 1 → **rotate**.

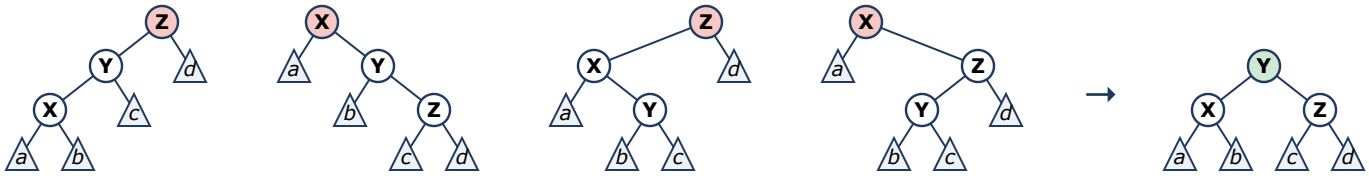


Example 18.27 Work bottom-up: 4 and 1 are leaves → 0. Node 3: 0 − 1 = −1. Node 5: 2 − 0 = **+2**. Root 2: 1 − 3 = **−2**. Not balanced.

18.7.3 Rotations

Walk from the bottom up, computing BFs; rotate at each node with |BF| > 1. Four cases:

Case	Imbalanced node	Its child	Fix
Left-Left (straight)	BF > +1	left child BF 0 or +1	**Right** rotation on the node (1 rotation)
Right-Right (straight)	BF < −1	right child BF 0 or −1	**Left** rotation on the node (1 rotation)
★ Left-Right (zigzag)	BF > +1	left child BF **−1**	Left rotate the **left child**, then right rotate the node (2 rotations)
★ Right-Left (zigzag)	BF < −1	right child BF **+1**	Right rotate the **right child**, then left rotate the node (2 rotations)



Right rotation on node *n* (left child *L*):

1. *n*'s left = *L*'s right.
2. *L*'s right = *n*.
3. Return *L* (new subtree root); the parent of *n* must now point to *L*.

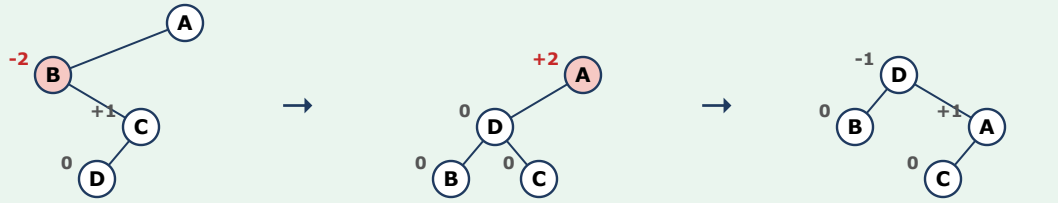
Left rotation on node *n* (right child *R*):

1. *n*'s right = *R*'s left.
2. *R*'s left = *n*.
3. Return *R*; the parent of *n* must now point to *R*.

```
Algorithm check_and_balance(Node* n):
  if balance_factor(n) > +1:
    if balance_factor(n->left) < 0:   // zigzag
      n->left = rotate_left(n->left)
    n = rotate_right(n)               // always
  else if balance_factor(n) < -1:
    if balance_factor(n->right) > 0:  // zigzag
      n->right = rotate_right(n->right)
    n = rotate_left(n)                // always
```

Memory aid: positive BF (left-heavy) → rotate **right**; negative (right-heavy) → rotate **left**. ★ Watch for a **sign change** between the node and its heavy-side child: that means a double rotation, child first.
A rotation just moves a few pointers → **Θ(1)**. `rotate_*` return the new subtree root so the parent can be reassigned.

Example 18.28: balance the stick A → B → C → D (B is A's left, C is B's right, D is C's left).

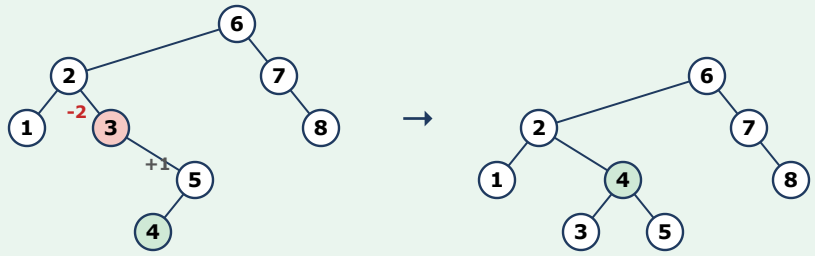


18.7.4 Inserting and removing

Insert: (1) normal BST insert. (2) Walk up from the new node checking BFs; fix the **first** imbalanced node (single or double rotation). ★ **Then stop: at most 2 rotations per insert.** The fix pulls the new node's long branch back up, so the subtree regains its old height and nothing above is affected.

Delete: (1) normal BST delete (successor or predecessor). (2) Walk up from the removed node's parent, fixing **every** imbalanced node **all the way to the root**. ★ **Up to Θ(log *n*) rotations:** each fix shortens a subtree, which can unbalance an ancestor whose shorter side that was.

Insert 4: lands as 5's left child. First imbalanced node going up = 3 (BF −2); its right child 5 has BF +1 (sign change, RL) → right on 5, left on 3. Done; 2 and 6 need no checks.



Deletion cascades. In the book's 33-node example, deleting 4 triggers right on 3 → left on 5 → left on 13 (3 rotations; up to 6 with double rotations). The smallest trees where one delete can force up to 2, 4, 6, 8 rotations have **4, 12, 33, 88** nodes, the minimum AVL sizes for heights 3, 5, 7, 9. Their root's children are minimum-size AVL trees of heights $h-2$ and $h-1$ (e.g. in the 12-node tree, deleting 1 triggers RR-4, RL-2, RR-10, RL-5). Why: deleting from the shorter child and rebalancing it shortens it to $h-3$, so the root needs ≤ 2 more rotations: `max_rot(h) = max_rot(h-2) + 2`, max_rot(2) = 0, max_rot(3) = 2 → 2, 4, 4, 6, 6, 8 for h = 4…9. Each new odd height adds one more possible (single or double) rotation.

Example 18.29 On an empty AVL tree: insert 17, 13, 14, 15, 16, delete 13. Double rotation = 2.

- 17, 13: fine. 14 → 17 is +2, 13 is −1 (LR) → left on 13, right on 17 → `14(13, 17)`. **2**
- 15 → left of 17, balanced. 16 → right of 15: 17 is +2, 15 is −1 (LR) → left on 15, right on 17 → `14(13, 16(15, 17))`. **2** (4 total)
- Delete 13 (leaf) → 14 is −2, 16 is 0 → single left on 14 → `16(14(_, 15), 17)`. **1**

Total: 5 rotations.

AVL operation	Best	Average	Worst
Find a value	Θ(1)	Θ(log *n*)	★ Θ(log *n*)
Insert / Delete a value	Θ(log *n*)	Θ(log *n*)	★ Θ(log *n*)

18.8 The STL Map and Set Containers

18.8.1 std::map<K, V> (<map>)

`std::map`: associative container of key-value `std::pair`s (`.first` = key, `.second` = value) with **unique keys kept in sorted order**. Ordering rules out hashing, so it is a **self-balancing BST** (usually a **red-black tree**: same Θ(log *n*) worst case as AVL).

Operation	`std::unordered_map`	`std::map`
Search / Insert / Delete key	Θ(1) average, Θ(*n*) worst	★ Θ(log *n*) average **and** worst

The tradeoff: keeping keys ordered costs logarithmic instead of constant average time.

```
std::map<std::string, double> food_prices;
food_prices.insert({"Apple", 3.99});
food_prices.insert({"Milk", 2.09});
food_prices.insert({"Tofu", 3.79});      // ...
std::cout << food_prices["Tofu"];        // 3.79 (BST lookup)

auto p = food_prices.lower_bound("Mango");  // first key ≥ "Mango"
std::cout << p->first << ' ' << p->second; // Milk 2.09
```

Same interface as `unordered_map`: `insert`, `emplace`, `try_emplace`, `operator[]`, `find`, `count`, `erase`, `size`, `empty`, `clear`.

`operator[]` on a missing key **inserts** it (value-initialized), just like `unordered_map`. Use `find()` when the key might not exist.

Member	Behavior
`insert(pair)` / `emplace(args...)`	Insert only if the key is absent. Return `pair<iterator, bool>` (element or the one that blocked it, inserted?).
`try_emplace(k, args...)`	Like emplace, but if `k` exists it does **not** move from the args (good for move-only values like `unique_ptr`). Key and value-constructor args are passed separately.
`find(k)`	Iterator to the element, or `end()`.
`count(k)`	0 or 1 (unique keys).
`erase(it)` / `erase(first, last)` / `erase(k)`	Iterator versions return the iterator after the last removed; key version returns the number removed (0 or 1).
`empty()`, `size()`, `clear()`	As usual.
`begin()` / `end()`	**Smallest key** / one past the largest. (`cbegin`/`cend` = const.)
`rbegin()` / `rend()`	**Largest key** / one before the smallest. ★ Ordered containers have **bidirectional** iterators (`--it`, reverse iteration); unordered ones don't.
`lower_bound(k)`	First element with key **not less than** `k` (≥ k), else `end()`.
`upper_bound(k)`	First element with key **greater than** `k` (> k), else `end()`.
`equal_range(k)`	`{lower_bound(k), upper_bound(k)}`.

★ `++it` moves to the next key in sorted order, so iterating `begin()` → `end()` is an **inorder traversal**.

Example 18.30: time-based key-value store. `set(key, value, t)`; `get(key, t)` returns the value most recently set at or before time `t`, or `""`. E.g. set fruit = apple @1, banana @5, cherry @7 → `get("fruit", 6)` = banana; `get("vegetable", 3)` = "" (vegetable first set @4).
Idea: per key, a **sorted** map timestamp → value; answer = the largest timestamp ≤ `t` = **the element just before `upper_bound(t)`**. If `upper_bound` is `begin()`, every timestamp is later → "". Keys need no order → outer `unordered_map`.

```
class TimedMap {
  std::unordered_map<std::string, std::map<int32_t, std::string>> time_map;
public:
  void set(const std::string& key, const std::string& value, int32_t timestamp) {
    time_map[key].emplace(timestamp, value);                    // O(log n)
  }
  std::string get(const std::string& key, int32_t timestamp) {
    const auto& m = time_map[key];
    auto it = m.upper_bound(timestamp);                         // O(log n)
    return it == m.begin() ? "" : std::prev(it)->second;
  }
};
```

If timestamps always arrive in increasing order, a `vector` per key stays sorted on its own: `set` becomes Θ(1) (push_back), `get` stays Θ(log *n*) (binary search).

18.8.2 std::set<K> (<set>)

`std::set`: **distinct** keys in **sorted** order, implemented as a self-balancing BST → Θ(log *n*) search/insert/erase. Beats the sorted-vector set (ch. 13), which needs Θ(*n*) shifting on insert/erase. Use it when you need a collection of keys *and* their order.

Members mirror `map`: `insert(k)` → `pair<iterator, bool>`; `insert(first, last)`; `insert({1, 3, 5, 7})` (initializer list); `empty`, `size`, `clear`; `begin`/`end`/`rbegin`/`rend` (smallest … largest); `erase(it)`, `erase(first, last)`, `erase(k)`; `find`, `count` (0/1); `lower_bound`, `upper_bound`, `equal_range`.

★ **Uniqueness comes from the comparator, not `operator==`.** `a` and `b` are equivalent iff `!(a < b) && !(b < a)`. If your comparator ignores some fields, distinct objects can count as duplicates and silently fail to insert.

Example 18.31: Discord rank manager. Rank users by total word count (words split by spaces). Needs: `post_message(user, msg)`, `get_word_count(user)` (0 if none), `get_user_range(lo, hi)` (all users with count in [lo, hi]), `get_highest_ranked_user()` (ties → lexicographically smaller name; "" if none).

- user → count: order doesn't matter → `unordered_map`.

- count → users: need ranges and the max → **sorted** `map`. Counts aren't unique, so map each count to a `set` of users (sorted, which also handles the tie-break).
- On each post: remove the user from its old count's set (erase the count if the set empties), add the words, insert into the new count's set.

```
class DiscordRankManager {
  std::unordered_map<std::string, int32_t> user_to_word_count_map;
  std::map<int32_t, std::set<std::string>> word_count_to_user_map;
public:
  void post_message(const std::string& user, const std::string& message) {
    std::stringstream msg_stream(message);
    std::string word;
    int32_t msg_word_count = 0;
    while (msg_stream >> word) ++msg_word_count;
    int32_t& total = user_to_word_count_map[user];
    std::set<std::string>& old_set = word_count_to_user_map[total];
    old_set.erase(user);
    if (old_set.empty()) word_count_to_user_map.erase(total);
    total += msg_word_count;
    word_count_to_user_map[total].insert(user);
  }
  int32_t get_word_count(const std::string& user) {
    auto it = user_to_word_count_map.find(user);
    return it == user_to_word_count_map.end() ? 0 : it->second;
  }
  std::vector<std::string> get_user_range(int32_t lo, int32_t hi) {
    std::vector<std::string> users;
    auto end = word_count_to_user_map.upper_bound(hi);
    for (auto it = word_count_to_user_map.lower_bound(lo); it != end; ++it)
      users.insert(users.end(), it->second.begin(), it->second.end());
    return users;
  }
  std::string get_highest_ranked_user() {
    if (user_to_word_count_map.empty()) return "";
    return *word_count_to_user_map.rbegin()->second.begin();   // max count, smallest name
  }
};
```

A first try with `map<int32_t, std::string>` is wrong: two users can share a word count.

✻ OPTIONAL · 18.8.3 std::multimap and std::multiset

`std::multimap` (<map>) and `std::multiset` (<set>): sorted, self-balancing BSTs that **allow duplicate keys**. Get all matches of a key with `equal_range(k)` → [first ≥ k, first > k). Unlike the unordered versions, **equal keys keep insertion order**: a duplicate always goes to the right of existing equal keys, so it is visited later.

```
std::multimap<std::string, int32_t> classes;
classes.insert({"EECS", 280}); classes.insert({"EECS", 183});
classes.insert({"EECS", 376}); classes.insert({"EECS", 281});
auto [lo, hi] = classes.equal_range("EECS");
for (auto it = lo; it != hi; ++it)
  std::cout << it->first << ' ' << it->second << '\n';
// EECS 280, EECS 183, EECS 376, EECS 281  (insertion order)
```

Streaming median with one multiset + one iterator `mid` (same Θ(log *n*) as the two-heap method of ch. 10). `mid` points at the median (odd size) or the **first** of the middle two (even size). On inserting `val`:

- Was empty → `mid` = the new element.
- Was **odd** and `val < *mid` → `--mid`; else no move.
- Was **even** and `val ≥ *mid` → `++mid`; else no move.

Median = `*mid` (odd) or avg(`*mid`, `*next(mid)`) (even). Stream 19 12 17 16 14 17 → medians 19, 15.5, 17, 16.5, 16, 16.5.

```
class MedianFinder {
  std::multiset<int32_t> data;
  std::multiset<int32_t>::iterator mid;
public:
  MedianFinder() : mid{data.end()} {}
  void add_value(int32_t val) {
    size_t prev_size = data.size();
```

```
    data.insert(val);
    if (prev_size == 0) mid = data.begin();
    else if (val < *mid && prev_size % 2 == 1) --mid;
    else if (val >= *mid && prev_size % 2 == 0) ++mid;
  }
  double find_median() {   // assumes at least one value added
    return data.size() % 2 == 1 ? *mid : static_cast<double>(*mid + *std::next(mid)) / 2;
  }
};
```

✻ OPTIONAL · 18.9 Tries: Trees for Efficient Prefix Matching (18.9.1–18.9.5)

18.9.1 Structure

Trie (re**trie**val tree, said "try"; a.k.a. **prefix tree**): each node holds a character; each root-to-node path spells a prefix, and a special end-of-word marker (* in diagrams) says a whole word ends there. Words with a common prefix share ancestors (e.g. "app" and "apple" share A-P-P). Used for set-of-strings lookup and fast prefix queries.

```
constexpr size_t CHAR_SIZE = 128;  // ASCII; 26 = letters
struct TrieNode {
  TrieNode* children[CHAR_SIZE] = {nullptr};
  bool is_end_of_word = false;     // the * marker
  ~TrieNode() {                    // recursive delete
    for (TrieNode* c : children) delete c;
  }
};
```

Children could also be an `unordered_map<char, TrieNode*>`, but that uses more memory than an array.

All operations are **Θ(*k*)** for a word of length *k*, usually treated as Θ(1) since word length is a small constant.

18.9.2 Insert

Walk down character by character, creating any missing node; mark the last node as end of word (e.g. "bean": B, E, A exist, so only N is created).

```
void Trie::insert(const std::string& word) {
  if (!root) root = new TrieNode;
  TrieNode* curr = root;
  for (char c : word) {
    if (!curr->children[c])
      curr->children[c] = new TrieNode;
    curr = curr->children[c];
  }
  if (!curr->is_end_of_word) {
    curr->is_end_of_word = true;
    ++sz;
  }
}
```

18.9.3 Search

Walk down; a missing character → false. If you reach the end, the answer is the last node's `is_end_of_word` (a prefix alone isn't a word).

```
bool Trie::contains(const std::string& word) const {
  TrieNode* curr = root;
  if (!curr) return false;
  for (char c : word) {
    if (!curr->children[c]) return false;
    curr = curr->children[c];
  }
  return curr->is_end_of_word;
}
```

18.9.4 Remove

A node can only be freed if no other word depends on it. Removing "bank" when "banker" exists just clears K's end-of-word flag. Removing "appearance" (no other word uses its tail) frees the trailing nodes. Steps: walk down saving each node on the path (fail if missing or not a word) → clear `is_end_of_word`, `--sz` → walk back up freeing nodes that are **not an end of word and have no children**, stopping at the first node still needed.

```
bool Trie::remove(const std::string& word) {
  if (!root) return false;
  TrieNode* curr = root;
  std::vector<TrieNode*> path(word.size(), nullptr);          // path[i] = node for word[i]
  for (size_t i = 0; i < word.size(); ++i) {
    if (!curr->children[word[i]]) return false;
    path[i] = curr = curr->children[word[i]];
  }
  if (!curr->is_end_of_word) return false;
  curr->is_end_of_word = false;
  --sz;
  for (size_t i = word.size(); i-- > 0; ) {                     // bottom-up
    TrieNode* node = path[i];
    if (node->is_end_of_word || has_valid_children(node->children, CHAR_SIZE)) break;
    delete node;
    (i == 0 ? root : path[i - 1])->children[word[i]] = nullptr;
```

```
  }
  return true;
}
bool Trie::has_valid_children(TrieNode* nodes[], size_t size) {   // any non-null child?
  for (size_t i = 0; i < size; ++i) if (nodes[i]) return true;
  return false;
}
```

Condensed from the book's version. Note: the book's loop returns `false` when it stops early at a still-needed node, even though the word *was* removed; stopping with `break` and returning `true` is the intended behavior.

18.9.5 Destructor

```
void Trie::clear() {   // ~TrieNode recurses
  delete root;         //   (postorder delete)
  root = nullptr;
  sz = 0;
}
size_t Trie::size() const { return sz; }
Trie::~Trie() { clear(); }
```

Applications: autocomplete / search suggestions, spell checkers, and **IP routing**: routers store destination prefixes in an optimized trie and walk the packet's address bits to find which port to forward it out of.

Summary of Complexities

Structure	Search	Insert	Delete	Notes
Binary tree (no ordering)	$\Theta(n)$	$\Theta(1)$ best, $\Theta(n)$ worst	$\Theta(n)$	Space: array $\Theta(n)$ to $\Theta(2^n)$; pointer $\Theta(n)$
BST	$\Theta(1)$ best · $\Theta(\log n)$ average · **$\Theta(n)$ worst** (stick)			Inorder = sorted, $\Theta(n)$
AVL tree	**$\Theta(\log n)$ worst** (search best $\Theta(1)$)			Insert ≤ 2 rotations; delete up to $\Theta(\log n)$
`std::map` / `std::set` (and multi-)	$\Theta(\log n)$ worst			Red-black tree, sorted, bidirectional iterators
`unordered_map` / `unordered_set`	$\Theta(1)$ average · $\Theta(n)$ worst			Hash table, no order (ch. 17)
Trie (✻)	$\Theta(k)$, k = word length			Prefix queries

Traversal	Order	Structure	Typical use
Preorder	P L R	Recursion	Root first; copy a tree; top-down work (invert)
Inorder	L P R	Recursion	Sorted order of a BST; reconstruction partition
Postorder	L R P	Recursion	Bottom-up: height, deletion/freeing, combine child answers
Level-order	by depth, left → right	Queue (BFS)	Per-level questions (use queue size)

All four traversals visit each node once: $\Theta(n)$ time.